A D PATEL INSTITUTE OF TECHNOLOGY

(Affiliated to CVM University), Vallabh Vidyanagar
Department of Information Technology

MINI PROJECT REPORT

on

STUDENT RESULT MANAGEMENT SYSTEM

(Python, Tkinter and SQLite)

Submitted by

Akmal Kadivar

Enrollment No.: 12402080601001

Semester V | Division IT1
Academic Year 2026-27

1. Abstract

The Student Result Management System is a desktop application developed in Python that allows an institute to enter, store, search and delete the examination results of students. The graphical user interface is built with the Tkinter library and all records are stored permanently in a local SQLite database. The system accepts the marks of three subjects (Python, Maths and Computer), validates them, and automatically calculates the total, percentage and grade of every student. The project replaces error-prone manual result preparation with a fast, accurate and easy-to-use software tool.

2. Introduction

Preparing results by hand is slow and error-prone. Marks have to be added, percentages worked out and grades assigned for every student, and a single arithmetic slip can change a student's grade. Searching for an old record in paper files or scattered spreadsheets also wastes time.

This project addresses these problems with a small, self-contained application. A staff member types the roll number, name and marks into a form, and the program computes the result instantly and saves it. All saved records are shown in a table that can be refreshed at any time, and any record can be searched or deleted using the roll number. No internet connection or separate database server is required.

3. Objectives

- To build a simple GUI through which student marks can be entered and managed.
- To calculate the total, percentage and grade automatically using a fixed grading scheme.
- To store the records permanently in a relational database (SQLite).
- To validate user input so that wrong or incomplete data is never saved.
- To provide search, delete and refresh operations on the stored records.

4. Technologies Used

Component	Technology	Purpose
Language	Python 3	Application logic, calculations and validation
GUI	Tkinter and ttk	Form, buttons, message boxes and the Treeview records table
Database	SQLite (sqlite3)	Permanent local storage of student records
Storage file	student_results.db	Created automatically on first run

Table 1: Technologies used in the project

5. System Design

5.1 Database Design

The application uses a single table named students. The roll number is the primary key, so two students can never share the same roll number. Every column is declared NOT NULL to guarantee that complete records are stored.

Field	Data Type	Description
roll_no	TEXT (Primary Key)	Unique roll number of the student
name	TEXT	Full name of the student
python_marks	INTEGER	Marks in Python (0 to 100)
maths_marks	INTEGER	Marks in Maths (0 to 100)
computer_marks	INTEGER	Marks in Computer (0 to 100)
total	INTEGER	Sum of the three subjects (out of 300)
percentage	REAL	Total divided by 3
grade	TEXT	Grade derived from the percentage

Table 2: Structure of the students table

5.2 Working of the System

The program follows a simple flow. The user enters the details in the form and clicks a button. The input is validated first. If it is valid, the total, percentage and grade are calculated, the record is inserted into the SQLite database, and the records table on the screen is refreshed. Search and delete operations use the roll number to find the record in the database.

6. Modules and Implementation

6.1 Input Validation

The validate_marks() function checks that the roll number and name are not empty, that all three marks are whole numbers, and that every mark lies between 0 and 100. If any rule fails, an error message box is displayed and nothing is saved.

6.2 Result Calculation

The calculate_result() function adds the three marks, divides the total by 3 to get the percentage, and assigns the grade as shown below.

```
def calculate_result(python_marks, maths_marks, computer_marks):
    total = python_marks + maths_marks + computer_marks
    percentage = total / 3
    if percentage >= 90:   grade = "A+"
    elif percentage >= 80: grade = "A"
    ...                    # B, C and D follow the same pattern
    else:                  grade = "F"
    return total, percentage, grade
```

6.3 Add Student

add_student() validates the input, calculates the result and inserts the record using a parameterised SQL query (INSERT with ? placeholders), which also protects against SQL injection. If the roll number already exists, SQLite raises an IntegrityError and the program shows a Duplicate Roll Number message instead of crashing.

6.4 Display, Search and Delete

- **display_students()** reads all records ordered by roll number and fills the Treeview table, with the percentage shown to two decimal places.
- **search_student()** fetches one record by roll number, fills the form fields with it and shows the total, percentage and grade in a message box.
- **delete_student()** finds the record, asks for confirmation using a Yes/No dialog, and only then removes it from the database.
- **clear_fields()** empties all input fields, and the Refresh button reloads the table from the database.

6.5 User Interface

The window has a title banner, a Student Details form with five input fields, a row of five colour-coded buttons (Add, Search, Delete, Clear and Refresh) and a scrollable Student Records table. Green is used for adding, blue for searching and red for deleting, so that the purpose of each button is clear at a glance. When the window is closed, the database connection is closed properly.

7. Grading Scheme

Percentage	Grade	Remark
90 and above	A+	Outstanding
80 to below 90	A	Excellent
70 to below 80	B	Very Good
60 to below 70	C	Good
50 to below 60	D	Satisfactory
Below 50	F	Fail

Table 3: Grade assigned for each percentage range

8. Testing

The system was tested with valid and invalid inputs to confirm that each rule works correctly.

Test Case	Expected Result	Status
Marks 95, 92, 98	Total 285, 95.00%, grade A+	Pass
Marks 72, 65, 80	Total 217, 72.33%, grade B	Pass
Mark entered as 105 or -5	Error: marks must be 0 to 100	Pass

Test Case	Expected Result	Status
Mark entered as text (e.g. abc)	Error: enter valid whole numbers	Pass
Empty roll number or name	Error: enter both fields	Pass
Same roll number added twice	Duplicate Roll Number message	Pass
Search a missing roll number	Not Found message	Pass
Delete an existing record	Confirmation, then record removed	Pass

Table 4: Test cases and results

9. Advantages and Limitations

Advantages

- Results are calculated instantly and without arithmetic mistakes.
- Data is stored permanently, so it is available the next time the program is opened.
- Simple interface that needs no technical training, and works offline.

Limitations

- Only three fixed subjects are supported and there is no update (edit) option.
- It is a single-user desktop application with no login or report printing.

10. Future Scope

The project can be extended with an Update option to correct marks, support for more subjects and semesters, user login for staff, export of results to PDF or Excel, and graphs showing class performance. It could also be moved to a web or networked database so that many users can work on the same data.

11. Conclusion

The Student Result Management System meets its objective of making result handling quick, accurate and organised. It combines Python logic, a Tkinter interface and an SQLite database into a complete application, and demonstrates input validation, database operations and event-driven programming. With the improvements listed in the future scope, it can grow into a full result management solution for an institute.

12. References

- Python Software Foundation, Python Documentation: https://docs.python.org/3/
- Python Documentation, tkinter - Python interface to Tcl/Tk: https://docs.python.org/3/library/tkinter.html
- Python Documentation, sqlite3 - DB-API 2.0 interface for SQLite databases: https://docs.python.org/3/library/sqlite3.html
- SQLite Documentation: https://www.sqlite.org/docs.html